



Panchip Microelectronics Co., Ltd.

PAN3029/3060 Series SDK User's Guide

Current Version. 0.6

Date of publication. 2024.04

Shanghai Panqi Microelectronics Co.

Address: Room 302, Building D, No. 666 Shengxia Road,
Zhangjiang Hi-Tech Park, Shanghai, China Tel: 021-50802371

Website: <http://www.panchip.com>

documentation

The contents of this document may be updated from time to time due to version upgrades or other reasons. Unless otherwise agreed, the contents of this document are intended as a guide to use only, and all statements, information and recommendations in this document do not constitute any warranty, express or implied.

logo

Panqi is a trademark of Panqi Microelectronics Ltd. Other names mentioned in this document are trademarks/registered trademarks of their respective owners.

statement denying or limiting responsibility

All or some of the products, services or features described in this document may be excluded from your purchase or use. Unless otherwise contractually agreed, Granite Microelectronics Ltd. makes no representations or warranties, express or implied, with respect to the contents of this document.

revision history (of a document, web page etc)

releases	revision time	descriptive
V0.1	2023.06	Initial Version Creation
V0.2	2023.08	1. Modify the operation flow after using the single packet mode 2. Add interface to get current channel energy detection
V0.3	2023.09	1. Updating of the transmitter power table 2. Updating the MAPM Interface
V0.4	2023.12	1. Updating the Transmit Power Table 1-23 Stages
V0.5	2023.12	1. Updating the SDK Porting Instructions
V0.6	2024.04	Modify the function interface and chip description

catalogs

1 Overview	1
2 SDK Usage Flow	2
2.1 Initialization Process	2
2.1.1 Interface Functions	2
2.1.2 Realization Functions	2
2.1.3 How to use	2
2.2 Parameter Configuration Flow	2
2.2.1 Interface Functions	2
2.2.2 Realization Functions	2
2.2.3 How to use	3
2.3 Sending Process	3
2.3.1 Interface Functions	3
2.3.2 Realization Functions	3
2.3.3 How to use	4
2.4 Receiving process	4
2.4.1 Interface Functions	4
2.4.2 Realization Functions	4
2.4.3 How to use the program	4
3 SDK Interface Description	5
3.1 rf_flag interface	5
3.2 rf_init	6
3.3 rf_sleep_wakeup	6
3.4 rf_deepsleep	7
3.5 rf_sleep	7
3.6 rf_get_tx_time	7
3.7 rf_set_mode	8
3.8 rf_get_mode	8
3.9 rf_set_tx_mode	9
3.10 rf_set_rx_mode	9
3.11 rf_set_rx_single_timeout	10
3.12 rf_get_snr	10
3.13 rf_get_rssi	11
3.14 rf_get_channel_rssi	11
3.15 rf_get_irq	12
3.16 rf_clr_irq	12
3.17 rf_set_refresh	13
3.18 rf_set_preamble	13
3.19 rf_cad_on	14
3.20 rf_cad_off	14

3.21	rf_set_syncword.....	15
3.22	rf_get_syncword.....	15
3.23	rf Early Interrupt Interface	16
3.24	rf_set_mapm_rx_on.....	16
3.25	rf_set_mapm_rx_off.....	16
3.26	rf_set_mapm_para	17
3.27	rf_receive.....	17
3.28	rf_enter_continuous_rx.....	18
3.29	rf_enter_single_timeout_rx.....	18
3.30	rf_enter_single_rx.....	19
3.31	rf_single_tx_data	19
3.32	rf_enter_continuous_tx/rf_continuous_tx_send_data.....	20
3.33	rf_set_agc	20
3.34	rf_set_dcdc_mode.....	21
3.35	rf_set_ldr	21
3.36	rf Smart Search Interface.....	22
3.37	get_chirp_time.....	22
3.38	check_cad_rx_inactive.....	23
4	SDK Porting.....	24
5	PAN3029/3060 Series Status Chart.....	25
6	Application Notes	27
6.1	Low power hibernation.....	27
6.2	CAD Functions	27
6.3	Sensitivity RSSI.....	27
6.4	DCDC Function	28
6.5	EFUSE Features	28
6.6	RF Parameter Description.....	30
6.7	Four-Wire SPI and FIFO.....	31
7	Annex to the report of the Secretary-General on the implementation of the Programme of Action for the Sustainable Development of the United Nations.....	33
7.1	Transmit Power Meter.....	33

1 summarize

This document mainly introduces the SDK interface functions of PAN3029/3060 series chips. It includes the initialization process of the chip and the process of sending and receiving data.

SDK related files are pan_rf.cpan_rf.hpan_port.cpan_port.hpan_param.h.

Confidential

2 SDK Usage Flow

This section describes in detail the basic flow of using the SDK, including the most basic initialization, parameter configuration and data sending and receiving.

2.1 initialization process

2.1.1 interface function

```
rf_init().
```

2.1.2 Implementation Functions

1. Wake up the chip from deep sleep state to standby3 state.
2. Load FT configuration
3. Initialize register configuration
4. AGC Configuration
5. Initialization chip TX_IO/RX_IO

2.1.3 Usage

Called when the chip is first powered up or wakes up from deepsleep.

2.2 Parameter Configuration Flow

2.2.1 interface function

```
rf_set_default_para().
```

2.2.2 Implementation Functions

1. Set the frequency, Code_Rate, bandwidth, spreading factor, transmit power and CRC

```
rf_set_freq(DEFAULT_FREQ);  
rf_set_code_rate(DEFAULT_CR);  
rf_set_bw(DEFAULT_BW); rf_set_sf(DEFAULT_SF);  
rf_set_crc(CRC_ON).  
rf_set_tx_power(DEFAULT_PWR).
```

2.2.3 Usage

After the chip has been initialized, the interface function is called in the **standby3** state.

2.3 Send process

2.3.1 interface function

```
rf_enter_continuous_tx(); rf_continuous_tx_send_data().  
rf_single_tx_data().
```

2.3.2 Implementation Functions

1. Switch to STB3 status

```
rf_set_mode(RF_MODE_STB3)
```

2. Open PA

```
rf_set_ldo_pa_on
```

3. Open TX_IO

```
rf_port.set_tx
```

4. Setting the launch mode

```
rf_set_tx_mode
```

5. Enter TX state and send data

```
rf_send_packet
```

6. Obtain the launch time of this package of data (optional)

```
rf_get_tx_time
```

2.3.3 Usage

The interface function is called after the chip has been initialized and configured with parameters. When the interface function is called and the chip has finished sending, the IRQ pin is set high and an interrupt is generated.

2.4 Receiving process

2.4.1 interface function

```
rf_enter_continuous_rx(); rf_enter_single_timeout_rx().  
rf_enter_single_rx().
```

2.4.2 Implementation Functions

1. Switch to STB3 status

```
rf_set_mode(RF_MODE_STB3)
```

2. Open RX_IO

```
rf_port.set_rx
```

3. Set the receiving mode

```
rf_set_rx_mode
```

4. Set up to enter the receiving state

```
rf_set_mode(RF_MODE_RX)
```

2.4.3 Usage

The interface function is called after the chip is initialized and the configuration parameters are completed. When data is received, IRQ pin is set high. Polling or interrupting can be used to determine if the packet has been received, and when it has, the packet collector function is called to remove the packet from the chip.

3

SDK Interface Description

This section describes in detail the user interface functions provided in the SDK.

Most of the parameter configurations need to be done after the initialization of the chip is complete (in `standby3` state).

3.1 rf_flag interface

syntax

```
int rf_get_rcv_flag(void)

void rf_set_rcv_flag(int status) int

rf_get_txmit_flag(void)

void rf_set_txmit_flag(int status)
```

goal

The IRQ interrupt generated after the chip is set to transmit or receive passes the event to the user via the `flag`. The user can use the polling method to determine the transmit or receive result of the chip.

parameters

Status	Current event status
RADIO_FLAG_IDLE	RADIO_FLAG_TXDONE
RADIO_FLAG_RXDONE	RADIO_FLAG_RXTIMEOUT
RADIO_FLAG_RXERR	RADIO_FLAG_PLHDRXDONE
RADIO_FLAG_MAPM	

3.2 rf_init

syntax

```
RF_Err_t rf_init(void)
```

goal

Initializes the chip, including wakeup, loading FT configuration, initializing register configuration, and AGC configuration. Called when the chip first powers up or wakes up from deepsleep.

return value

FAIL operation failed

OK. - OK. The operation was successful.

3.3 rf_sleep_wakeup

syntax

```
RF_Err_t rf_sleep_wakeup(void)
```

goal

Sets the chip to wake up from **sleep** state.

Used when the chip is in **sleep** state and needs to be woken up.

When the chip is in **sleep** state, the register values of the chip will not be reset, and the parameter configurations will still be the same as before sleep after wake-up.

return value

FAIL operation failed

OK. - OK. The operation was successful.

3.4 rf_deepsleep

syntax

```
RF_Err_t rf_deepsleep(void)
```

goal

Reduces power consumption by switching the chip from **standby3** state to **deep sleep** state.

return value

FAIL operation failed

OK. - OK. The operation was successful.

3.5 rf_sleep

syntax

```
RF_Err_t rf_sleep(void)
```

goal

Reduces power consumption by switching the chip from the **standby3** state to the **sleep** state.

return value

FAIL operation failed

OK. - OK. The operation was successful.

3.6 rf_get_tx_time

syntax

```
uint32_t rf_get_tx_time(uint8_t size)
```

goal

Get the time required for this packet to be transmitted, a demonstration of this function usage is provided within `rf_single_tx_data()`.

parameters

Size: the length of the packet for this launch

return value

The time in us required for this packet launch.

3.7 rf_set_mode

syntax

```
RF_Err_t rf_set_mode(uint8_t mode)
```

goal

Set the state in which the chip works, including deepsleep/sleep/standby1/standby2/standby3/tx/rx, the parameter configuration is usually done in standby3 state.

parameters

Mode	The state in which the chip operates
RF_MODE_DEEP_SLEEP	
RF_MODE_SLEEP RF_MODE_STB1	
RF_MODE_STB2 RF_MODE_STB3	
RF_MODE_TX	
RF_MODE_RX	

return value

FAIL operation failed

OK. - OK. The operation was successful.

3.8 rf_get_mode

syntax

```
uint8_t rf_get_mode(void)
```

goal

Reads the current operating status of the chip, including deepsleep/sleep/standby1/standby2/standby3/tx/rx.

return value

chip status value

3.9 rf_set_tx_mode

syntax

```
RF_Err_t rf_set_tx_mode(uint8_t mode)
```

goal

Set the firing mode, which can be set to single firing and continuous firing.

parameters

Mode	Transmit Data Mode
RF_TX_SINGLE	
RF_TX_CONTINUOUS	

return value

FAIL operation failed

OK. - OK. The operation was successful.

3.10 rf_set_rx_mode

syntax

```
RF_Err_t rf_set_rx_mode(uint8_t mode)
```

goal

Set the reception mode, which can be set to single reception, timeout reception, and continuous reception.

parameters

Mode	reception
	mode
RF_RX_SINGLE	
RF_RX_SINGLE_TIMEOUT	
RF_RX_CONTINUOUS	

return value

FAIL operation failed

OK. - OK. The operation was successful.

3.11 rf_set_rx_single_timeout

syntax

```
RF_Err_t rf_set_rx_single_timeout(uint16_t timeout)
```

goal

Sets the timeout for timeout reception, valid only in timeout reception mode.

parameters

Timeout	1~65535	Timeout in ms
---------	---------	---------------

return value

FAIL operation failed

OK. - OK. The operation was successful.

3.12 rf_get_snr

syntax

```
float rf_get_snr(void)
```

goal

Reads the value of the signal-to-noise ratio SNR, higher SNR means better signal quality. This interface needs to be called when a packet is received, and before the RxDone interrupt is cleared. If the interrupt is cleared, the value is invalidated, as is the RSSI.rf_irq_process() provides a demonstration of this function's usage.

Pseudocode example:

```
// RxDone IRQ received
snr = rf_get_snr(); rssi=
rf_get_rssi();
rf_receive(buf).
```

return value

SNR value

3.13 rf_get_rssi

syntax

```
int8_t rf_get_rssi(void)
```

goal

The interface needs to be called when a packet is received and before the RxDone interrupt is cleared. the value is invalidated if the interrupt is cleared. the same is true for SNR. rf_irq_process() function provides a demonstration of its usage. If the interrupt is cleared, the value is invalidated, as is the case with SNR. A demonstration of the use of this function is provided within the rf_irq_process() function. More usage can be found in the RSSI Application Reference document.

Example:

```
// RxDone IRQ received
snr = rf_get_snr(); rssi=
rf_get_rssi();
rf_receive(buf).
```

return value

RSSI value

3.14 rf_get_channel_rssi

syntax

```
int8_t rf_get_channel_rssi(void)
```

goal

The PAN3029/3060 supports the channel energy detection function, and the channel rssi value can be used to indicate the amount of energy under the channel. When the chip enters the RX state, it can call this interface to get the channel detection energy and read the value of channel energy strength.

Example:

```
//After the chip enters the RX state
channel_rssi= rf_get_channel_rssi();
```

return value

RSSI value

3.15 rf_get_irq

syntax

```
uint8_t rf_get_irq(void)
```

goal

Read the interrupt register value. Used rf_irq_process.

return value

Interruption results

3.16 rf_clr_irq

syntax

```
RF_Err_t rf_clr_irq(uint8_t flags)
```

goal

Clear the corresponding interrupt.

parameters

Flags	interrupt mode
REG_IRQ_MAPM_DONE	


```
REG_IRQ_RX_PLHD_DONE REG_IRQ_RX_DONE
REG_IRQ_CRC_ERR REG_IRQ_RX_TIMEOUT
REG_IRQ_TX_DONE
```

return value

FAIL operation failed

OK. - OK. The operation was successful.

3.17 rf_set_refresh

syntax

```
RF_Err_t rf_refresh(void)
```

goal

Register parameter refresh. Does not modify register values.

return value

FAIL operation failed

OK. - OK. The operation was successful.

3.18 rf_set_preamble

syntax

```
RF_Err_t rf_set_preamble(uint16_t reg)
```

goal

Setting the Chip Transmit Pilot Code is only valid for the transmitter and needs to be configured before transmitting the data. When the data is launched, the launch content of "leading code+ data ", the longer leading code represents the longer time for the whole launch content to send out the packet.

parameters

Reg	8-65535	Transmit Pilot Code Length
-----	---------	----------------------------

return value

FAIL operation failed

OK. - OK. The operation was successful.

3.19 rf_cad_on

syntax

```
RF_Err_t rf_cad_on(uint8_t threshold, uint8_t chirps)
```

goal

Setting the chip to turn on the CAD function, which needs to be configured before the chip enters receive mode, modifies the receive threshold.

parameters

Threshold Receive Threshold
CAD_DETECT_THRESHOLD_0A
CAD_DETECT_THRESHOLD_10
CAD_DETECT_THRESHOLD_15
CAD_DETECT_THRESHOLD_20

chirps Symbol detection number
CAD_DETECT_NUMBER_1 CAD_DETECT_NUMBER_2
CAD_DETECT_NUMBER_3

return value

FAIL operation failed

OK. - OK. The operation was successful.

3.20 rf_cad_off

syntax

```
RF_Err_t rf_cad_off(void)
```

goal

Set the chip to turn off the CAD function and bring back the receive threshold.

return value

FAIL operation failed

OK. - OK. The operation was successful.

3.21 rf_set_syncword

syntax

```
RF_Err_t rf_set_syncword(uint8_t sync)
```

goal

Sets the synchronization word for the chip.

parameters

Sync	Sync
------	------

return value

FAIL operation failed

OK. - OK. The operation was successful.

3.22 rf_get_syncword

syntax

```
uint8_t rf_get_syncword(void)
```

goal

Reads the synchronization word of the chip.

return value

synchronized character value

3.23 rf Early Interrupt Interface

syntax

```
RF_Err_t rf_set_plhd(uint8_t addr, uint8_t len) void  
rf_set_plhd_rx_off(void)  
uint8_t rf_get_plhd_len(void)  
RF_Err_t rf_plhd_receive(uint8_t *buf, uint8_t len)
```

goal

The Early Interrupt function is to view the data that has been solved during the process of reading a frame of data by the chip, to judge whether it is what you want or not, and then decide whether to continue to read or abandon the frame of data. For more information, please refer to the PAN3029_3060 Series Early Interrupt Application Reference Document.

3.24 rf_set_mapm_rx_on

syntax

```
void rf_set_mapm_on(void)
```

goal

Enable mapm mode, turn on mapm interrupt allowance

return value

not have

3.25 rf_set_mapm_rx_off

syntax

```
void rf_set_mapm_off(void)
```

goal

Off mapm mode, off mapm interrupts allowed

return value

not have

3.26 rf_set_mapm_para

syntax

```
void rf_set_mapm_para(stc_mapm_cfg_t *p_mapm_cfg)
```

goal

Setting mapm mode related parameters

parameters

p_mapm_cfg->fn	Used to configure the number of Fields in the mapm preamble.
p_mapm_cfg->fnm	Used to configure the number of repetitions per Field in mapm preamble
p_mapm_cfg->gfs	Used to configure the last Group in a Field and its ADDR location
function selection p_mapm_cfg->gn	Used to configure the number of Groups in a Field
p_mapm_cfg->pgl	Used to configure the number of Preamble in the first Group
p_mapm_cfg->pgn	Used to configure the number of Preamble in other Groups
p_mapm_cfg->gn	Number of chirps before syncword after all Fields have been sent.

return value

not have

3.27 rf_receive

syntax

```
uint8_t rf_rcv_packet(uint8_t *buff)
```

goal

Receive a packet of data

parameters

Buff Used to receive packets

return value

Received packet length

3.28 rf_enter_continuous_rx

syntax

```
RF_Err_t rf_enter_continuous_rx(void)
```

goal

Set the chip to enter continuous reception mode reception.

In this mode, the chip will remain in the receive state, and when data is received, the chip will generate an rx IRQ related interrupt and continue to receive.

return value

FAIL operation failed

OK. - OK. The operation was successful.

3.29 rf_enter_single_timeout_rx

syntax

```
RF_Err_t rf_enter_single_timeout_rx(uint32_t timeout)
```

goal

Sets the chip to enter timeout receive mode reception.

In this mode, the chip will receive once, when receiving data, the chip will generate rxdone IRQ interrupt and wait for the user's subsequent operation. When no data is received within the timeout period, the chip will generate rxtimeout IRQ interrupt and wait for the user's subsequent operation. If the user gives up the reception, the chip needs to exit the reception state voluntarily, otherwise, the chip will start a timeout reception again. After use, the user's software needs to be switched to STB3 to exit the receiving state first, and then carry out subsequent operations.

parameters

Timeout Timeout

return value

FAIL operation failed

OK. - OK. The operation was successful.

3.30 rf_enter_single_rx

syntax

```
RF_Err_t rf_enter_single_rx(void)
```

goal

Set the chip to enter single receive mode reception.

In this mode, the chip will perform one reception, when the data is received, the chip will generate rx IRQ related interrupt and exit the reception state, the user software needs to switch to STB3 to exit the reception state first, and then carry out the subsequent operation.

return value

FAIL operation failed

OK. - OK. The operation was successful.

3.31 rf_single_tx_data

syntax

```
RF_Err_t rf_single_tx_data(uint8_t *buf, uint8_t size, uint32_t *tx_time)
```

goal

Sets the chip to enter single-transmit mode and transmit data.

In this mode, the chip will perform a launch, when the launch is completed, the chip will generate txdone IRQ interrupt and exit the launch state (the operating current is reduced), the user's software needs to be switched to STB3 to exit the launch state before subsequent operations.

parameters

Buf Send buff

Size Packet length

Tx_time Get the transmission time of this packet, tx_time parameter is not necessary, users can delete it according to their needs.

return value

FAIL operation failed

OK. - OK. The operation was successful.

3.32 rf_enter_continuous_tx/rf_continuous_tx_send_data

syntax

```
RF_Err_t rf_enter_continuous_tx(void)
```

```
RF_Err_t rf_continuous_tx_send_data(uint8_t *buf, uint8_t size)
```

goal

Sets the chip into continuous transmit mode and transmits data.

In this mode, after the chip has finished transmitting once, the chip generates a txdone IRQ interrupt and remains in the transmitting state (the operating current is still the transmitting current but no data is transmitted) and waits for the user's subsequent operation.

Subsequently, the user can choose to stop transmitting and switch the chip state to standby3; or choose to continue transmitting by directly calling rf_continuous_tx_send_data(), and continue transmitting without repeating the rf_enter_continuous_tx() operation.

parameters

Buf Send buff

Size Packet length

return value

FAIL operation failed

OK. - OK. The operation was successful.

3.33 rf_set_agc

syntax

```
RF_Err_t rf_set_agc(bool NewState)
```


goal

Configure the chip AGC. configure and turn on the AGC when you initialize the chip by default.

parameters

State	AGC_OFF/ AGC_ON
-------	-----------------

return value

FAIL operation failed

OK. - OK. The operation was successful.

3.34 rf_set_dcdc_mode

syntax

```
RF_Err_t rf_set_dcdc_mode(uint8_t dcdc_val)
```

goal

Sets the chip's DCDC mode, which is off by default. DCDC mode results in lower receive current.

Before turning on this function, please contact and confirm whether the module supports this function to turn on, otherwise it is easy to cause damage to the module.

parameters

Dcdc_val	Schema Type
DCDC_ON	
DCDC_OFF	

return value

FAIL operation failed

OK. - OK. The operation was successful.

3.35 rf_set_ldr

syntax

```
RF_Err_t rf_set_ldr(uint32_t mode)
```

goal

Sets the chip LDR low rate mode, which is off by default. The low rate mode can further reduce the transmission rate to get better transmission effect.

parameters

Mode	Mode Type
	LDR_ON
	LDR_OFF

return value

FAIL operation failed

OK. - OK. The operation was successful.

3.36 rf Smart Search Interface

syntax

```
RF_Err_t rf_set_auto_sf_tx_preamble(int sf, int sf_range[], int size, int chirp_counts[]) RF_Err_t rf_set_auto_sf_rx_on(int sf_range[], int size)
RF_Err_t rf_set_auto_sf_rx_off(void)
int calculate_chirp_count(int sf_range[], int size, int chirp_counts[])
```

goal

Intelligent search function can realize intelligent identification of SF parameters in the channel during reception to achieve the purpose of receiving different SF signal data. Please refer to the "PAN3029_3060 Series_Intelligent Search Application Reference Document" for more information about the function.

3.37 get_chirp_time

syntax

```
uint32_t rf_get_chirp_time(uint8_t bw, uint8_t sf)
```

goal

Calculates the duration of a single chirp.

parameters

SF: SF_5/SF_6/SF_7/SF_8/SF_9/SF_10/SF_11/SF_12

BW: BW_62_5K/BW_125K/BW_250K/BW_500K

return value

Individual chirp duration in us.

3.38 check_cad_rx_inactive

syntax

```
bool check_cad_rx_inactive(uint32_t one_chirp_time)
```

goal

Use CAD to detect the current channel status for channel detection before CAD reception.

parameters

one_chirp_time: duration of a single chirp

return value

LEVEL_ACTIVE The channel is active and should continue to receive

LEVEL_INACTIVE No signal on the channel, reception should be aborted

4 SDK Porting

This section describes the SDK porting process and notes in detail. keil configure floating point hardware as "Not Used", configure "One ELF Section per Function" unchecked, set the optimization level to "default". " is unchecked and the optimization level is set to "default". After compiling the code, the Flash size required by the SDK is 9125 (8.91kB) and the SRAM size required is 1485 (1.45kB).

The SDK "pan_port.c" provides the interface functions that the user needs to port. To run the SDK correctly, the following interfaces need to be implemented:

- spi_cs_set_high(), SPI chip select pull high.
 - spi_cs_set_low(), SPI chip select pull low.
 - spi_readwritebyte(), SPI data transfer.
 - rf_delay_ms(), rf_delay_us(), delay function.
 - RX_IO/TX_IO implementation, chip transceiver IO switching interface, including rf_antenna_init, rf_antenna_tx, rf_antenna_rx, rf_antenna_close.
- RX_IO/TX_IO implementation, SDK through the SPI read/write registers to realize this part, through the SPI control chip internal

GPIO4 and GPIO3 implement RX_IO/TX_IO.

Users can also use external MCU IO to control RX_IO/TX_IO, the specific implementation depends on the design of the base board.

- TCXO_IO implementation, chip active crystal IO control interface, including rf_tcxo_init, rf_tcxo_close.
- TCXO_IO implementation, SDK through the SPI read and write registers to achieve this part, through the SPI control chip internal GPIO5

Implement TCXO_IO.

Users can also use external MCU IO to control TCXO_IO, depending on the PAN3029/3060 series module version and MCU board design.

5 PAN3029/3060 Series Status Chart

This chapter describes the state switching diagram of PAN3029/3060 series chips, which is convenient for users to understand the working principle of the chips. Among them, the wake-up process and configuration initialization from DeepSleep to STB3 (standby3) corresponds to the `rf_init` function in SDK; STB3 to DeepSleep corresponds to the `rf_deepsleep` function in SDK; Sleep to STB3 switching process corresponds to the `rf_sleep_wakeup` function in SDK; STB3 to Sleep corresponds to the `rf_sleep` function in SDK; STB3 to Sleep corresponds to the `rf_sleep` function in SDK; and STB3 to Sleep corresponds to the `rf_sleep` function in SDK. Sleep to STB3 switching process corresponds to `rf_sleep_wakeup` function in SDK; STB3 to Sleep corresponds to `rf_sleep` function in SDK.

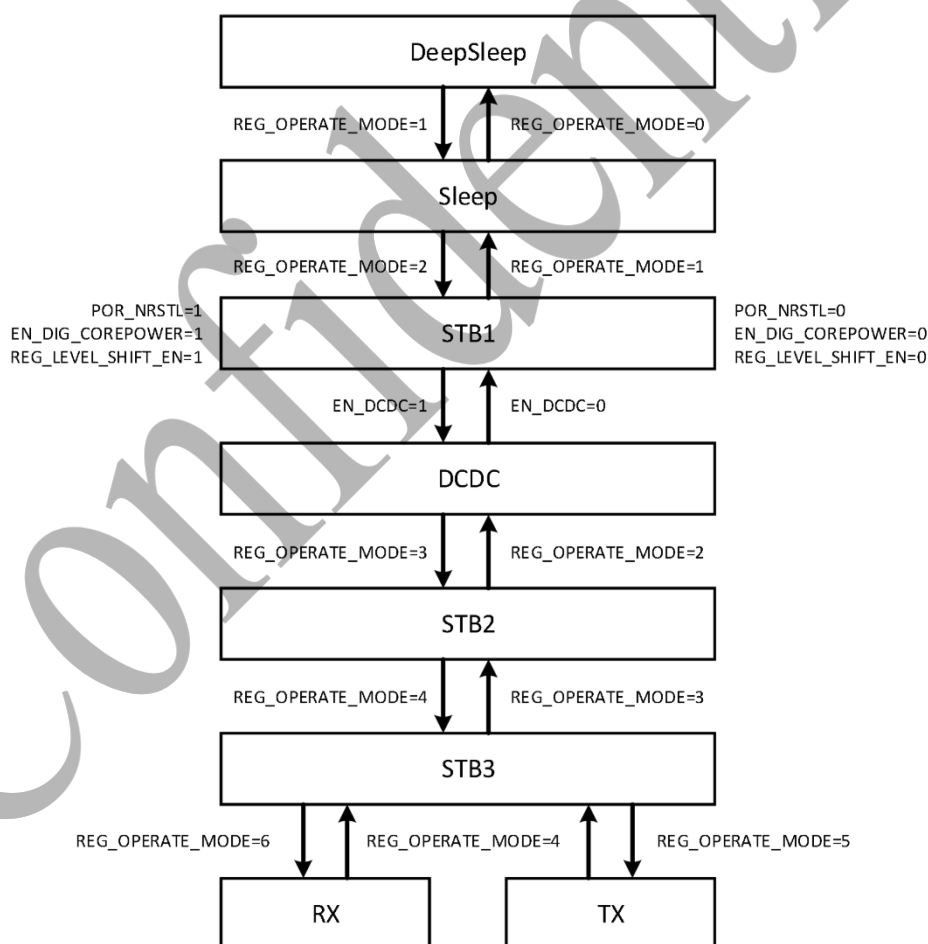


Figure 6-1 PAN3029 Power-Up Procedure

The PAN3029/3060 series chips have seven states after power-up, including DeepSleep, Sleep, LDO operation (STB1), OSC operation (STB2), OSC output (STB3), TX mode, and RX mode. All states can be switched through the configuration registers.

Note: If the peripheral circuit of the chip does not support DCDC, you may not configure EN_DCDC and enter STB2 directly from STB1.

The DCDC function is not used by default. If you need to use the DCDC function, please refer to the corresponding application documentation.

Confidential

6 Application Notes

6.1 Low power hibernation

For low-power hibernation, four interface functions are provided in the SDK.

One is sleep **Sleep** mode. Includes the `rf_sleep` hibernation function from STB3 to **Sleep**, and the `rf_sleep` hibernation function from **Sleep** to STB3.

`rf_sleep_wakeup` Wakeup function.

One is **DeepSleep** mode. It contains `rf_deepsleep` hibernation function from STB3 to **DeepSleep** and `rf_init` wakeup function from **DeepSleep** to STB3. Note that when PAN3029/3060 series chips enter **DeepSleep** and wake up again, the original configuration parameters need to be reconfigured and the transceiver mode needs to be selected and configured again.

6.2 CAD Functions

The CAD function, which modifies the reception threshold of the chip, may affect the reception sensitivity of the chip. The CAD function needs to be turned off after use.

Please refer to the PAN3029_3060 Series_CAD Application Reference Document for more function descriptions.

6.3 Sensitivity RSSI

Reading the signal strength value needs to be called when a packet is received and before clearing the `rxdone` interrupt. If the interrupt is cleared, the value will be invalidated. The measurement range of RSSI is from -12 to -110, which is slightly different for different parameter (SF, BW) modes.

It is recommended to turn on AGC when you need to read the RSSI of the chip, which can improve the accuracy of the RSSI of the chip. Please refer to "PAN3029_3060 Series_RSSI Application Reference Document" for more function description.

6.4 DCDC Function

The DCDC function is turned on to reduce the receive current of the chip to 4.77mA, and is turned off by default. an interface function is provided in the SDK to turn the DCDC function on or off.

```
RF_Err_t rf_set_dcdc_mode(uint8_t dcdc_val)
```

Parameter dcdc_val

DCDC_ON opens

DCDC_OFF clotur
e

The DCDC function can be enabled in STB1~STB3 and RX modes.

After turning on the DCDC function, when the chip needs to send, you need to manually turn off the DCDC function in advance.

Before turning on the DCDC function, please contact and confirm whether the module supports this function, otherwise it is easy to cause damage to the module.

6.5 EFUSE Features

The PAN3029/3060 series chips support 82Bytes of eFuse space area for customer's use, which can be used to store data that will not be lost when power is lost. There are four interface functions provided in the SDK for using the eFuse function.

```
uint32_t rf_efuse_on(void)
```

goal

Setting the parameters for
turning on the eFuse function
on the chip

not have

return value

FAIL operation failed

OK. - OK. The operation was successful.

```
uint32_t rf_efuse_off(void)
```

goal

Setting the chip to disable the eFuse function

Para

meter

None

return value

FAIL operation failed

OK. - OK. The operation was successful.

uint8_t rf_efuse_read_byte(uint8_t reg_addr, uint8_t efuse_addr)

goal

Read the specified efuse address to store the data parameters.

reg_addr efuse Entry address, customer area default

0x3c efuse_addr efuse Internal address, customer area 0x2d~0x7f return value

value Reads the data returned by the specified efuse address.

void rf_efuse_write_byte(uint8_t reg_addr, uint8_t efuse_addr, uint8_t value)

goal

Write the data parameters you want to store to the specified efuse address.

reg_addr efuse entry address, customer area default

0x3c efuse_addr efuse internal address, customer area 0x2d~0x7f value Data to be written to efuse

Return

Value

None

eFuse reading and writing requires special attention:

1. The eFuse can only be burned once, and once burned, it cannot be modified.
2. You cannot write to the same address of the eFuse more than once, as this may cause damage to the eFuse device.
3. The voltage range for writing the eFuse is 2.25V to 2.75V, with a typical value of 2.5V, so configuring the eFuse, it is recommended that the external

VDD1 uses 2.. The read voltage will normally use .

4. The eFuse only supports read and write operations in STB3, do not read or write in other modes.

5. The eFuse area does not support sequential read/write operations for SPI and I2C.

6. The maximum clock rate for eFuse reads and writes is 8MHz.

6.6 RF Parameter Description

➤ Channel bandwidth (BW)

The channel bandwidth is the lower and upper frequency limits of the signal allowed to pass through the channel, and can be interpreted as a frequency passband with values in the range of 62.5 KHz, 125 KHz, 250 KHz, and 500 KHz. e.g., if a channel has a permissible passband of 1.5 kHz to 15 kHz, then the bandwidth is 13.5 kHz. Increasing the BW can increase the effective data rate in order to shorten the transmission time, but at the expense of some of the receiver sensitivity, thus affecting the communication distance. transmission time, but at the expense of some receiver sensitivity, which affects the communication distance.

➤ Spreading factor (SF)

The value of SF ranges from SF5-12, the smaller the value, the higher the transmission rate and the shorter the transmission distance, on the contrary, the larger the value of SF, the slower the transmission rate and the longer the transmission distance. The larger the SF value, the slower the transmission rate and the farther the transmission distance. When the same amount of data is transmitted, the larger the SF value, the longer the transmission time.

➤ Coding rate (CR)

CR is the ratio of the useful (non-redundant) portion of the data stream, ranging from 1 to 4, corresponding to 1=4/5, 2=4/6, 3=4/7, and 4=4/8. In other words, if the coding rate is k/n, the encoder generates a total of n bits of data for every k bits of useful information, of which n-k are redundant. The larger the CR, the more redundant data is transmitted in a single transmission, which results in a lower effective data rate and longer transmission time. The larger CR is, the more excess data is transmitted in a single transmission, the lower the effective data rate and the longer the transmission time.

➤ Checksum (CRC)

The CRC is set at the TX end and not at the RX end. the RX end receives the information in the signal and reads the CRC in the frame structure to decide whether or not to perform a CRC check.

➤ low speed mode

The value range is 1 or 0, when it is 1, the low rate mode is enabled, and the default is off, when it is enabled, the communication rate will be reduced, the reception sensitivity will be increased by 1-2dBm, and the communication distance will be slightly increased.

➤ Number of leading code symbols

Value range 8-65535, default value is 8. When realizing the Wake-on-Air application scenario, the number of leading code symbols can be increased, and the transmit time will also be increased, so as to realize the function of the receiver being woken up by the long leading code in the low-power mode.

Note: The above RF parameters need to be configured according to the actual application scenarios, and the RF configuration of the transceiver needs to be the same in order to communicate normally. the selection of RF data transmission time and parameters can be referred to the calculation tool in the SDK.

6.7 Four-wire SPI and FIFO

External devices can access the registers and FIFOs on the chip via four-wire SPI for configuration.

The PAN3029/3060 series chips implement a Slave Slave of the SPI bus for reading and writing registers and FIFOs. The SPI bus is a four-wire system, respectively:

- SCK (clock)
- CSN (chip select signal, active low)
- MOSI (data input)
- MISO (data output)

Among them, SCK, CSN, MOSI are controlled by the host Master, and MISO is controlled by the Slave.

The communication process starts with CSN level low and ends when CSN level is high. Host Master

Data is sent via MOSI and received by MISO. data is generated on the falling edge of SCK and sampled on the rising edge.

The information transmitted by the Master consists of two parts: Address Byte and Data Byte. The first 7 bits of Address Byte are address bits `addr`; the last 1 bit is read/write bit `wr`, which is set to 1 for write operation and 0 for read operation.

SPI has three transmission modes:

- Single: Single byte transmission mode. The message is only 2 byte, Master sends Address Byte through MOSI, if it is write operation, Master continues to send Data Byte through MOSI; if it is read operation, Master reads Data Byte replied by Slave on MISO.
- Burst: Burst continuous transmission mode. The message is larger than 2 bytes, Address Byte is followed by several Data Bytes, there is no need to add Address Bytes between Data Bytes, the slave will automatically increment the address between each Data Byte internally. CSN signal is pulled high after the last Data Byte, and stays low for the rest of the message transmission.
- FIFO: FIFO read/write mode. In this mode, single byte or continuous transmission can be realized, and the transmission rules are the same as those of Single mode and Burst mode, the difference is that the address bit `addr` in the Address Byte can only be configured as 7'h1, and the Slave does not do the address increment operation between Data Bytes.

The SPI write timing is as follows:

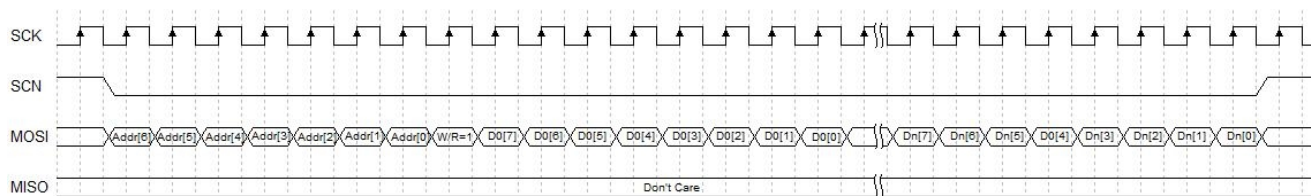


Figure 7-1 SPI Write Timing

The SPI read timing is as follows:

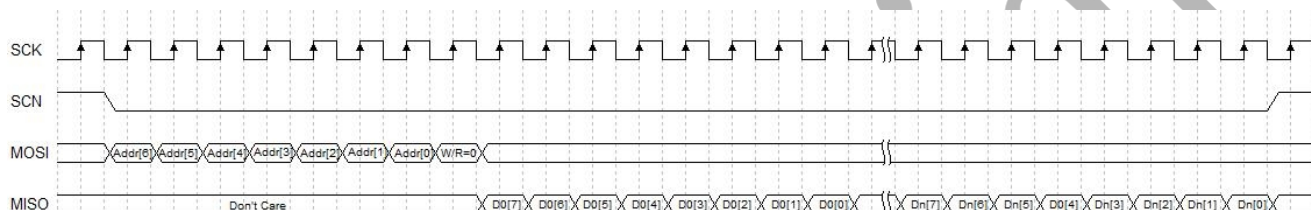


Figure 7-2 SPI Read Timing

The PAN3029/3060 series chips have a 255bytes FIFO for storing TX module transmit data and RX module decode data.

FIFO of single-port RAM, which can only realize the storage and reading of single packet data information. In the case that a packet of data is already stored in FIFO, the packet data should be read before writing, otherwise the previous packet of data in FIFO will be overwritten.

7 attachment (email)

7.1 transmission power meter

档位 \ 频率	410MHz		430MHz		450MHz		460MHz	
	功率(dBm)	电流(mA)	功率(dBm)	电流(mA)	功率(dBm)	电流(mA)	功率(dBm)	电流(mA)
1	-18.733	7.267	-18.165	7.418	-18.767	7.546	-19.679	7.607
2	-8.276	9.42	-7.936	9.563	-8.596	9.671	-9.548	9.758
3	1.599	17.935	1.748	18.349	0.947	18.725	-0.111	18.932
4	3.925	20.301	4.513	20.661	4.114	20.967	3.348	21.439
5	4.921	21.76	5.329	22.183	4.79	22.662	3.865	23.076
6	5.413	22.933	5.501	23.385	4.776	23.811	3.841	24.136
7	7.104	26.605	7.611	26.904	7.345	27.468	6.805	28.54
8	7.655	28.062	8.152	28.282	7.951	28.995	7.491	30.298
9	8.638	30.074	9.185	30.663	8.755	31.328	8.117	32.387
10	8.966	32.23	9.29	31.92	9.106	32.679	8.897	34.62
11	10.274	35.676	10.746	36.083	10.571	37.273	10.298	39.578
12	10.816	39.495	10.988	38.391	10.753	38.94	10.56	41.211
13	11.567	41.972	11.815	41.256	11.612	42.165	11.468	44.97
14	12.83	46.616	13.205	46.748	13.077	48.484	12.919	52.149
15	13.782	50.643	14.252	51.595	14.182	54.016	14.001	58.272
16	14.871	55.72	15.404	57.538	15.36	60.671	15.015	64.89
17	15.404	58.355	15.957	60.647	15.797	63.599	15.317	67.146
18	16.052	63.254	16.602	65.475	16.522	69.186	16.267	74.527
19	16.493	66.892	16.956	68.616	16.899	72.499	16.738	78.608
20	17.414	73.954	17.868	75.84	17.8	80.261	17.632	86.882
21	18.229	87.976	18.297	85.286	17.994	86.519	17.694	90.391
22	19.436	102.128	19.414	98.008	19.07	98.768	18.736	102.895
23	19.754	105.378	19.771	101.687	19.433	102.835	19.105	107.278

Note: The power block is an argument to the `rf_set_tx_power` function. Other bands will have slightly different transmit power and transmit current.